

What is Encryption

- Encryption is a security technique used to **protect data** by transforming it from a readable form (plaintext) to an unreadable, scrambled form (ciphertext).
- Only someone who has the **correct cryptographic key** can convert the ciphertext back to its original form.
- **Detailed Purpose**
 - **Confidentiality:** Prevents unauthorized access
 - **Data Protection:** Secures sensitive information (passwords, banking details, personal data)
 - **Secure Communication:** Ensures safe data transfer over insecure channels (internet, wireless networks)
 - **Integrity & Trust:** Prevents tampering and builds trust in digital systems

Encryption is the foundation of **modern cybersecurity**, used everywhere — from WhatsApp messages to online banking, ATM transactions, and website security (HTTPS).

Concept of Keys

A **key** in cryptography is a special piece of information that controls the encryption and decryption process.

- A key decides **how** plaintext is converted into ciphertext.
- Without the correct key, the encrypted data is meaningless.
- Keys must be protected because anyone with the key can unlock the information.

Why Keys Are Important

- They ensure secrecy even if someone captures the encrypted data.
- The same algorithm (AES, RSA, etc.) can be used globally, but *security lies in the key*, not the algorithm.
- Different encryption systems use keys differently → leads to two major types of encryption.

A. Public Key (The "Mailbox Slot")

- **Visibility:** This key is **known to everyone**. You can send it to anyone via email or post it on a website.
- **Function:** It is used to **Encrypt** a message.
- **Analogy:** Think of a public mailbox on the street. Anyone can walk up and drop a letter (data) into the slot (encrypt). But once it's inside, no one standing on the street can reach in and take it back out.

B. Private Key (The "Master Key")

- **Visibility:** This key is **kept secret** by the owner. It must never be shared.
- **Function:** It is used to **Decrypt** a message.
- **Analogy:** This is the key the mailman has. Only the person with this specific key can open the mailbox and read the letters (decrypt) that were dropped in by the public.

Types of Encryptions

Encryption is classified into **two main types** based on the relationship between the encryption and decryption keys.

1. Symmetric Encryption (Detailed): Symmetric encryption uses **one single key** for both encryption and decryption.

- The same shared secret key must be known by both sender and receiver.
- Very fast because operations are simple and optimized.
- Best for encrypting large amounts of data (files, databases, streaming data).

Strengths

- High performance and speed.
- Efficient even on low-power devices.
- Good for real-time data encryption.

Limitations

- **Key distribution problem** — How do two parties exchange the key securely?
- If the shared key is stolen or leaked, the entire communication becomes compromised.

Examples: AES (Advanced Encryption Standard), DES (Data Encryption Standard), Blowfish

2. Asymmetric Encryption: Asymmetric encryption uses **two mathematically related keys**:

- **Public Key** → shared openly, used for encryption
- **Private Key** → kept secret, used for decryption
- Anyone can encrypt data using the public key.
- Only the owner of the private key can decrypt the data.
- Solves the key exchange problem — no need to share private keys.

Strengths

- High security; even if the public key is known, the private key cannot be guessed.
- Enables digital signatures, secure key exchange, and authentication.
- Essential for secure internet communication (SSL/TLS).

Limitations

- Slower than symmetric encryption.
- Not suitable for encrypting very large files directly. (Usually used for encrypting symmetric keys, not bulk data.)

Examples: RSA

Feature	Symmetric Encryption	Asymmetric Encryption
Key Type	One shared secret key	Two keys: Public + Private
Key Usage	Same key encrypts & decrypts	Public key encrypts, private key decrypts
Key Distribution	Difficult — both parties must share the key secretly	Easy — public key can be shared openly
Speed	Very fast and efficient	Slower due to complex mathematics
Security Dependence	Depends entirely on protecting the secret key	Depends on secrecy of private key; public key is safe to share
Scalability	Poor for large networks — each pair of users needs a separate key	Excellent — one public key works for everyone
Use Cases	Bulk data encryption, secure storage, internal systems	Digital signatures, secure authentication, SSL/TLS, email security
Vulnerability	Key leakage compromises everything	Even if public key is leaked, security remains intact
Examples	AES, DES, RC4, Blowfish	RSA, ECC, Diffie–Hellman

Difference Between Hashing and Encryption

Hashing: Hashing is a one-way process that converts data into a **fixed-length hash value** using a mathematical function. It **cannot be reversed** to get the original data. It is mainly used for **integrity verification** and **password storage**.

Encryption: Encryption is a two-way process that converts plain text into **cipher text** using a key. It **can be reversed** (decryption) using the correct key. It is used to **protect confidentiality** of data during storage or transmission.

Point	Hashing	Encryption
1. Definition	One-way transformation of data into a fixed hash value.	Two-way transformation of plaintext to ciphertext.
2. Reversibility	Irreversible (cannot get original data).	Reversible using decryption key.
3. Purpose	Ensures data integrity, password verification.	Ensures data confidentiality and secure communication.
4. Output Size	Fixed-length output (e.g., 256-bit).	Variable-sized ciphertext depending on algorithm and data.
5. Key Usage	No keys; uses hash functions (SHA-256, MD5).	Uses keys (symmetric or asymmetric).
6. Vulnerabilities	Susceptible to collisions if weak hash is used.	Susceptible to brute-force or key theft.
7. Example Use	Password storage, file integrity check.	Secure messaging, SSL/TLS, data encryption.
8. Algorithms	MD5, SHA-1, SHA-256, BLAKE2.	AES, DES, RSA, ECC.

Cipher

- A **cipher** is a **method or algorithm used to convert plaintext into unreadable ciphertext** to protect the confidentiality of information. It defines the rules for **substitution, permutation, and transformation** of data during encryption and decryption.
- In simple terms, a cipher is the **technique or formula** that scrambles data so only authorized users with the correct key can read it.

Types of Ciphers

1. **Substitution Cipher** – Replaces characters with other characters (e.g., Caesar cipher).
2. **Transposition Cipher** – Rearranges the position of characters without altering them.
3. **Block Cipher** – Encrypts data in fixed-size blocks (e.g., AES).
4. **Stream Cipher** – Encrypts data one bit/byte at a time (e.g., RC4).

Example: Caesar Cipher: shifting each letter by a fixed number (A→D, B→E, etc.).

Transposition Cipher

- A **transposition cipher** is an encryption technique in which the **positions of characters are rearranged** according to a specific rule or key, but the characters themselves remain unchanged.
- It hides the message by **permuting the order** of the plaintext instead of substituting characters.
- Examples: Rail Fence cipher, Columnar Transposition cipher.

Encrypt a Plain Text Using a Transposition Cipher Key

Example: Columnar Transposition Cipher

- **Plaintext:** ATTACK TONIGHT (remove spaces → ATTACKTONIGHT)

- **Key: 3 1 4 2** (This key shows the order of columns)

Step 1: Write plaintext in rows using 4 columns (because key length = 4)

Col1	Col2	Col3	Col4
A	T	T	A
C	K	T	O
N	I	G	H
T	X	X	X

(X is padding to fill the matrix)

Step 2: Read columns in the order of the key

- Key: **3 1 4 2**
- So read **column 3** → **1** → **4** → **2**
- **Final Ciphertext: TTGXACNTAOHXTKIX**

Substitution Cipher

- A **substitution cipher** is an encryption technique where **each character in the plaintext is replaced with another character, symbol, or number** according to a fixed rule or key. The characters are **substituted**, but their positions remain the same.
- It protects data by replacing original symbols with new ones, making the message unreadable without the key.

Types of Substitution Ciphers

1. **Caesar Cipher:** Shifts each letter by a fixed number. Example: Shift 3 $\Rightarrow$ A $\rightarrow$ D, B $\rightarrow$ E, C $\rightarrow$ F.
2. **Monoalphabetic Cipher:** Each plaintext letter maps to a unique ciphertext letter (random substitution).
3. **Polyalphabetic Cipher:** Uses multiple substitution alphabets with a keyword.
4. **Playfair Cipher:** Encrypts digraphs (pairs of letters) instead of single letters.
5. **Hill Cipher:** Uses matrix multiplication to substitute blocks of letters.

Example: Simple Substitution (Caesar Shift 3)

Plaintext: ATTACK

Ciphertext: DWWDNFN (A $\rightarrow$ D, T $\rightarrow$ W, C $\rightarrow$ F, K $\rightarrow$ N)

Block Cipher

- A **block cipher** is a symmetric encryption technique that **divides plaintext into fixed-size blocks** (such as 64-bit or 128-bit) and encrypts each block using the **same key**. Each block is processed as a whole unit through multiple rounds involving substitution, permutation, mixing, and key addition.
- Block ciphers ensure confidentiality by transforming each block of data into ciphertext of the **same size**.

Key Features of Block Ciphers

- Process data in fixed-size blocks, not byte-by-byte
- Use a single symmetric key for encryption and decryption
- Work through repeated rounds of substitution & permutation
- Output block size equals input block size
- Provide high security and resistance against brute-force attacks

Common Block Sizes

- 64-bit (DES)
- 128-bit (AES, Blowfish in some modes)

Examples of Block Ciphers

- DES (Data Encryption Standard)
- AES (Advanced Encryption Standard)
- Blowfish / Twofish
- IDEA
- Camellia

Feistel Cipher

- A **Feistel Cipher** is a symmetric block cipher structure used to build many encryption algorithms (like DES).
- It converts plaintext into ciphertext through **multiple rounds of substitution and permutation**, but the design allows **easy decryption using the same process**.
- **The key idea:** In each round, the data block is split into Left (L) and Right (R) halves, and only the Right half is processed through a round function.

Step-by-Step Breakdown

1. **Encryption Step:** In each round (i), the new halves (L_{i+1} , R_{i+1}) are calculated from the old halves (L_i , R_i) using a Subkey (K_i):

$$L_{i+1} = R_i$$
$$R_{i+1} = L_i \oplus F(R_i, K_i)$$

1. **Decryption Step:** The incredible feature of the Feistel structure is that decryption is the exact same process. You run the ciphertext through the same rounds, but you use the Subkeys in reverse order (K_n , K_{n-1} , ..., K_1).

Why Feistel Cipher is Important

- **Same structure for both encryption & decryption:** The Feistel design allows decryption by simply reversing the round keys. No need to invert the round function.
- **Strong Security:** Confusion + diffusion achieved by repeated rounds.
- **Flexible Design:** Any function can be used for $f()$, even non-invertible ones.

EMAIL SECURITY

Steps to Analyse E-Mail Application Security Vulnerabilities

1. **Identify Assets and Data Flow:** Document how emails, attachments, login credentials, and user data are sent, stored, and processed.
2. **Check Authentication & Access Controls:** Verify password policies, multi-factor authentication, session management, and protection against unauthorized logins.
3. **Evaluate Encryption Mechanisms:** Ensure emails use secure protocols like **SSL/TLS**, verify if data is encrypted in transit and at rest, and check certificate validity.
4. **Test for Phishing & Spoofing Weaknesses:** Analyze whether the application prevents email spoofing using SPF, DKIM, and DMARC.

5. **Inspect Input Validation:** Check for vulnerabilities such as **XSS, SQL injection, command injection**, and malicious attachment handling.
6. **Analyze Attachment Handling:** Verify scanning of attachments, file-type restrictions, sandboxing, and protection against malware-infected files.
7. **Review Mail Server Configuration:** Check server security settings, open ports, misconfigurations, outdated software, and SMTP relaying issues.
8. **Check Logging & Monitoring:** Ensure proper logging of suspicious login attempts, email forwarding rules, and unauthorized access.
9. **Evaluate Spam & Malware Filters:** Review effectiveness of spam filters, blacklisting, heuristic analysis, and anti-malware engines.
10. **Perform Penetration Testing:** Simulate attacks (brute-force, replay attacks, header injection) to identify exploitable vulnerabilities.

Email Security Measures

1. **Learn to Inspect Message Headers:** Email headers show the sender IP, mail servers, and authentication results. Inspecting them helps detect spoofing, forged emails, and suspicious origins.
2. **Avoid Clicking on Links or Downloading Attachments:** This is a user-side precaution to prevent phishing and malware infections that spread through malicious links or attachments.
3. **Update Your DMARC Records With the Domain Registrar:** DMARC works with SPF and DKIM to stop attackers from sending fake emails using your domain. Updating DMARC improves protection against spoofing.
4. **Test Your SMTP Server:** Testing helps detect open relays, misconfigurations, weak authentication, or outdated software that attackers can abuse to send spam or steal email data.
5. **Make Use of SMTP SSL/TLS Ports:** Using SMTP ports with SSL/TLS (like 465 or 587) encrypts outgoing emails so attackers cannot intercept or read them during transmission.
6. **Deploy End-to-End Encryption for Maximum Email Security:** End-to-end encryption (e.g., PGP, S/MIME) ensures only the sender and receiver can read the message. Even email providers cannot decrypt it.:
7. **Use TLS With IMAP and POP3:** IMAP/POP3 retrieve emails from the server. Enabling TLS (IMAPS/POP3S) encrypts this communication and prevents password or email theft.
8. **Maintain IP Blacklists to Block Targeted Spams:** Blocking known malicious IPs prevents repeated spam, brute-force login attempts, and phishing emails from reaching inboxes.

Protocols Used for E-mail Security

1. PGP

- **PGP (Pretty Good Privacy)** is a popular data encryption and decryption computer program that provides cryptographic privacy and authentication for data communication.
- It is most commonly used for signing, encrypting, and decrypting **e-mails**, files, directories, and whole disk partitions.

PGP is unique because it uses a **Hybrid Cryptosystem**. It combines the best features of both Symmetric and Asymmetric encryption.

- **Symmetric Encryption (Speed):** Used to encrypt the actual message contents (because it is fast).
- **Asymmetric Encryption (Security):** Used to encrypt the *key* that locked the message (because it allows safe key sharing).

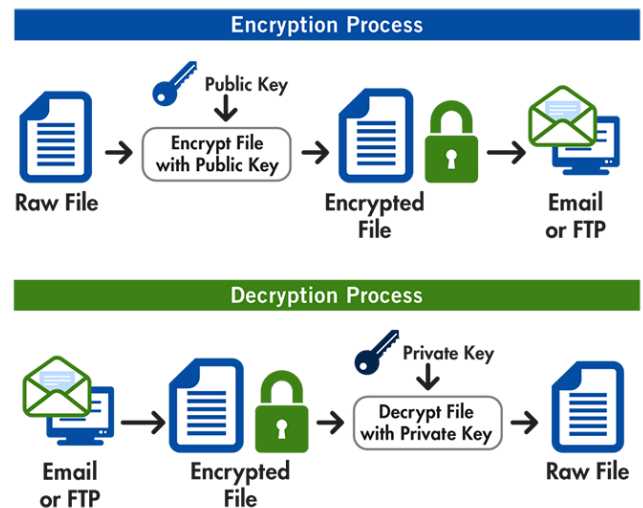
The Five Principal Services of PGP

1. **Authentication:** Ensures the message was created by the person claiming to send it. **Mechanism:** It uses a **Digital Signature** (hashing the message and encrypting the hash with the Sender's Private Key).
2. **Confidentiality:** Ensures unauthorized users cannot read the message. **Mechanism:** It creates a temporary, one-time **Session Key** to encrypt the message. Then, it encrypts that Session Key with the Receiver's Public Key.
3. **Compression:** PGP compresses the message (using ZIP-style algorithms) *after* signing but *before* encrypting. **Benefit:** Saves bandwidth and strengthens security (removes patterns in the plaintext that cryptanalysts look for).
4. **E-mail Compatibility (Radix-64 Conversion):** Encrypted data is binary (gibberish symbols), but many email systems only accept text (ASCII). **Mechanism:** PGP converts the binary encrypted block into a stream of printable ASCII characters (Radix-64) so it can travel through email servers safely.
5. **Segmentation:** If a message is too large for an email server to handle, PGP breaks it down into smaller parts and reassembles them at the destination.

The image shows how **public-key cryptography** is used to secure files during email or FTP transfer. It demonstrates **two major processes**:

1. Encryption Process (Sender Side)

- **Raw File:** The sender starts with the original file that needs to be protected.
- **Encrypt with Public Key:** The sender uses the **receiver's public key** to encrypt the raw file. Public key → used for encryption.
- **Encrypted File:** After encryption, the file changes into an unreadable ciphertext form.
- **Send through Email or FTP:** The encrypted file is safely sent over communication channels like email or FTP. Even if intercepted, it cannot be read.



2. Decryption Process (Receiver Side)

- **Receive Encrypted File** The receiver gets the encrypted file from email or FTP.
- **Decrypt with Private Key** The receiver uses **their private key** to decrypt the file. Private key → used for decryption.
- **Raw File Restored** After decryption, the original raw file is recovered and readable again.

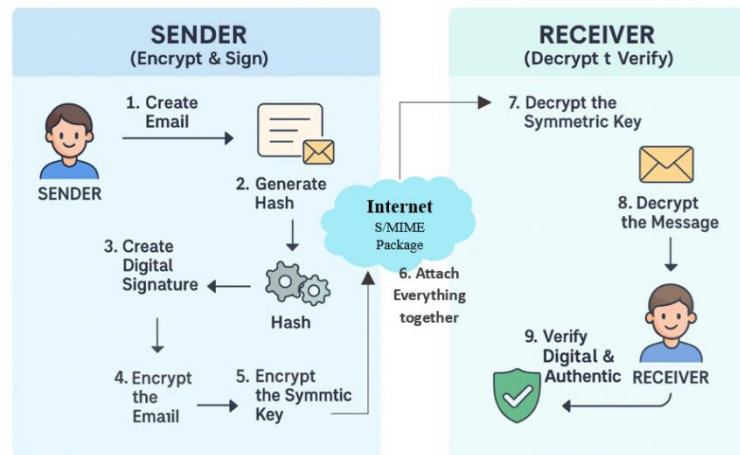
2. S/MIME

S/MIME (Secure/Multipurpose Internet Mail Extensions) is an email security protocol that provides **encryption, digital signatures, authentication, integrity, and non-repudiation** using public key cryptography.

1. Provides end-to-end encryption (only sender & receiver can read).
2. Compatible with Outlook, Gmail (enterprise), Apple Mail.
3. **Encryption:** Encrypts the email content using a symmetric key.
4. **Digital Signature:** Ensures authentication and non-repudiation.
5. **Integrity:** Hash functions ensure message is not modified.
6. **Key Management:** Handles certificates, trust chains, and revocation.

Steps:

1. **Create the Email** Sender writes the email message.
2. **Generate Hash of the Message** A hash (digest) is created to protect integrity.
3. **Create Digital Signature** The hash is encrypted using the sender's private key to form the signature.
4. **Encrypt the Message** The actual email is encrypted using a symmetric key (fast encryption).
5. **Encrypt the Symmetric Key** The symmetric key is encrypted using the receiver's public key.
6. **Attach Everything Together**
Encrypted message + encrypted symmetric key + digital signature = S/MIME email package.
7. **Receiver Decrypts the Symmetric Key** Receiver uses their private key to unlock the symmetric key.
8. **Decrypt the Message** Using the decrypted symmetric key, the email content is restored.
9. **Verify Digital Signature**
Receiver uses sender's public key to check the signature and confirm authenticity.



Advantages

- Strong security through PKI
- Trusted certificate-based authentication
- Widely supported in enterprise systems

Limitations

- Requires certificates
- Not easy for personal use
- Depends on CA infrastructure

3. TLS for SMTP, IMAP, POP3

TLS (Transport Layer Security) is a protocol that provides **secure, encrypted communication** between email clients and servers for sending and receiving emails. **Protocols Covered**

- SMTP + TLS → Secures outgoing mail
- IMAP + TLS → Secures retrieving email from server
- POP3 + TLS → Secures downloading emails from server

Working

- TLS establishes a secure handshake between client and server.
- Certificates authenticate the server (and optionally the client).
- A symmetric session key is generated for encrypted communication.

Security Functions

1. **Confidentiality:** Encrypts the email during transmission.
2. **Authentication:** Verifies the identity of the email server.
3. **Integrity:** Prevents tampering by verifying message blocks.
4. **Protection Against MITM Attacks:** Secures the path between sender & server.

Advantages

- Prevents eavesdropping
- Protects login credentials
- Easy to deploy on servers
- Works with all mail providers

Limitations

- Encrypts only “transit”, not end-to-end
- Server must support TLS
- Certificates must be properly configured

Firewall

A **firewall** is a security device or software that monitors, filters, and controls network traffic between **trusted internal networks** and **untrusted external networks**.

It prevents unauthorized access and protects systems from attacks.

Working of a Firewall (Simple Points)

1. **Traffic Monitoring:** Firewall continuously watches all incoming and outgoing packets.
2. **Rule Matching:** Every packet is compared with predefined security rules.
3. **Decision Making: Allow or Block:**
 - If packet matches an “allow” rule → traffic is permitted
 - If packet matches a “deny” rule → traffic is dropped
4. **Logging & Alerts:** Suspicious or blocked packets are logged for administrators.
5. **Types of filtering used:**
 - Packet filtering
 - Stateful inspection
 - Proxy filtering
 - Application layer filtering
6. **Protection:** Firewall blocks unauthorized access, malware, port scanning, DoS attempts, etc.

Packet Filtering in a Firewall

- **Packet filtering** is a firewall technique that controls network traffic based on **IP address, port number, and protocol**.
- It checks each incoming and outgoing packet against a set of rules and either **allows or blocks** it.
- **Example:** Allow packets from 192.168.1.10, block all traffic to port 23 (Telnet).
- Works at Network layer (Layer 3)
- Uses simple rule-based filtering
- Fast but limited (no deep inspection)